Table of Contents

Quick Start

The example code provided in this quick start guide is for educational and demonstration purposes only. It may not represent best practices for production use. This quick start was last updated for **UXR.QuestCamera v4.1.0**.

Breaking Changes Notice

If you've just updated the package to v4.0.0 or later, please re-read this guide for information on breaking changes from v3.1.3, as v4 is a complete rewrite of the package. For details regarding specific APIs, please check the reference manual.

> **IMPORTANT**
>
> The `Yield()` extension used in v3 has been removed. All `IAsyncDisposable`s **MUST** be awaited (`await DisposeAsync()` or `await using`). Do not call them from a fire-and-forget coroutine.

Setup

Dependencies

UXR.QuestCamera uses an AAR plugin to access the Camera2 API and requires the **External Dependency Manager for Unity (EDM4U)** package to handle native dependencies. If you have Firebase or Google SDKs in your project, you likely have it installed. If not, you can see the installation steps here: https://github.com/googlesamples/unity-jar-resolver?tab=readme-ov-file#getting-started

Unity Version Compatibility

This package uses some `Awaitable` methods for switching between threads for frame processing. Since `Awaitable` was only added in Unity 6, you will have to install **com.utilities.async** by Stephen Hodgson on older versions of Unity. You can see the installation steps here: https://github.com/RageAgainstThePixel/com.utilities.async

Android Manifest Setup

> **TIP**
>
> You can skip this step if you're using the Meta XR Core SDK v81 or higher by enabling the 'Enabled Passthrough Camera Access' setting in your `OVR Manager` instance and regenerating your AndroidManifest using the SDK's tools.

Add the following to your project's `AndroidManifest.xml` file:

```
<uses-feature android:name="android.hardware.camera2.any" android:required="true"/>
<uses-permission android:name="horizonos.permission.HEADSET_CAMERA"/>
```

The `HEADSET_CAMERA` permission is required by Horizon OS for apps to access the headset cameras. You must request it at runtime before using any of this package's APIs, like so:

```
if (!Permission.HasUserAuthorizedPermission(QuestCameraManager.HeadsetCameraPermission))
    Permission.RequestUserPermission(QuestCameraManager.HeadsetCameraPermission);
```

> **NOTE**
>
> If you are trying to access Meta's Avatar cameras instead of passthrough cameras, you should request `QuestCameraManager.AvatarCameraPermission` (`android.permission.CAMERA`) instead and add the appropriate `uses-permission` tag to your Manifest.

Gradle Template Setup

Since this package aims to use the latest stable version of the Camera2 package, your app's launcher must be compiled against API level 36 or higher. This is not an issue if your app's Target API Level is 36 (Android 16) or higher. If you cannot update your Target API Level, like due to Meta's API Level 34 requirement, you can use this workaround:

- Under Player -> Publishing Settings check "Custom Launcher Gradle Template" if it is unchecked.
- Open the `launcherTemplate.gradle` file created in Assets -> Plugins -> Android, and change this line:

```
compileSdk **APIVERSION**
```

to

```
compileSdk 36
```

> **NOTE**
>
> If you're on Unity 6000.4.4 or higher, you must also update the `compileSdk` property in your project's `mainTemplate.gradle` file, which is in the same folder as `launcherTemplate.gradle`. `mainTemplate.gradle` should be auto-generated by EDM4U. You can also manually create it by checking "Custom Main Gradle Template" under Player -> Publishing Settings.

Usage

Device Support

The Passthrough Camera API is restricted to the Quest 3 family and newer devices on Horizon OS version >= 74. Check support with `QuestCameraManager.Support`.

The property returns `PCASupport.Unsupported` on all non-Android platforms *and* **confirmed** unsupported Quest devices, `PCASupport.Supported` on Android **only** if the device is confirmed to support the API, and `PCASupport.Unknown` on Android if the Meta XR Core SDK is not included in your project. If you do not use the Meta XR Core SDK, it's recommended to manually confirm support.

Choosing the Camera

`QuestCameraManager` allows you to access `CameraInfo` objects and create `CameraDevice` objects.

The package provides a prefab which includes the script and references the built-in conversion shader. It's a persistent singleton, so add it to the first scene it's referenced in so it can be used in any scene loaded thereafter.

`QuestCameraManager` creates a `CameraInfo` object for each camera the native plugin detects. Each `CameraInfo` then exposes the supported resolutions, supported stream use cases, intrinsic information, and more of the physical camera device. You can get a camera associated with the left or right eye using `QuestCameraManager.TryGetDevice(CameraInfo.CameraEye, out CameraInfo)`. Filter by `CameraInfo.CameraEye.Unknown` to get the Avatar Camera.

If the device state changes, you can force the manager to query the hardware again and update the cached `CameraInfo` list by calling `QuestCameraManager.RefreshDevices()`.

> **NOTE**
> Even though `CameraInfo` is an `IDisposable`, `QuestCameraManager` manages the disposal of the objects returned by the above APIs. You can get independently managed `CameraInfo` objects using `QuestCameraManager.GetDevices()`.

Opening the Camera

You can open a camera device using `QuestCameraManager.OpenCamera(cameraInfo)`. This method returns a `CameraDevice`.

`CameraDevice` allows you to create capture pipelines, which provide actual images from the camera. It takes a bit for the camera device to open, so you need to wait for it using `CameraDevice.WaitForInitialization()` (Coroutine) or `CameraDevice.WaitForInitializationAsync()` (Task).

The async task returns a boolean confirming that the camera opened successfully (`State == ResourceState.Valid`). When using the coroutine method, check the `State` property after yielding. To get specific error reasons, check logcat or add listeners to `CameraDevice.OnDeviceDisconnected` and `CameraDevice.OnDeviceErred`.

If the camera couldn't open, release its native resources by awaiting `CameraDevice.DisposeAsync()`.

Creating a Capture Session

After opening a camera device, you can start a capture pipeline.

You can create two kinds of capture pipelines: continuous and on-demand. A continuous pipeline streams a sequence of frames to Unity, each converted from YUV to RGBA. If you don't need a live feed, you can save resources by using an on-demand capture pipeline.

To create a new continuous pipeline, use `CameraDevice.CreateContinuousPipeline(resolution)`. To create an on-demand pipeline, use `CameraDevice.CreateOnDemandPipeline(resolution)`. Supported resolutions for the camera are exposed in `CameraInfo.SupportedResolutions` as an array of Unity's `Resolution` objects. The last value in the array is usually the highest resolution.

These methods return `CapturePipeline<ContinuousCaptureSession>?` and `CapturePipeline<OnDemandCaptureSession>?` objects respectively. Each contains the session object (`CapturePipeline<T>.Session`) and a YUV-to-RGBA texture converter (`CapturePipeline<T>.Converter`).

As with `CameraDevice`, wait for the pipeline to open using `Session.WaitForInitialization()` or `Session.WaitForInitializationAsync()`, and check `State == ResourceState.Valid` when using the coroutine method. If the pipeline could not be started successfully, release its native resources by awaiting `CapturePipeline<T>.DisposeAsync()`.

Once started, you'll get frames from the camera in an ARGB32 `RenderTexture` accessible via `CapturePipeline<T>.Converter.Texture`. The latest timestamp is in `Converter.CaptureTimestamp`, and you can subscribe to

`Converter.OnFrameProcessed` for a callback. For on-demand pipelines, call `Session.TryRequestCapture(out var error)` when you need a new frame.

When you're done with the pipeline, dispose of it by awaiting `CapturePipeline<T>.DisposeAsync()`. This disposes both the converter and capture session simultaneously.

If you're also done using the `CameraDevice`, you can actually dispose of it before disposing the pipeline, as recommended by Android.

Capture Template

The optional `template` parameter maps directly to Android Camera2 capture templates. It provides the HAL with a baseline configuration. See the official documentation for the available values and guidance: https://developer.android.com/reference/android/hardware/camera2/CameraDevice#TEMPLATE_PREVIEW

Note that only a subset of all Camera2 capture templates are supported by the package.

Stream Use Case

The optional `streamUseCase` parameter is a Camera2 performance hint that allows the camera HAL to optimize for your workload. Check `CameraInfo.SupportedStreamUseCases` to see what the device reports. See the official documentation: https://developer.android.com/reference/android/hardware/camera2/params/OutputConfiguration#setStreamUseCase(long)

Graphics Format

The optional `textureFormat` parameter overrides the Unity `GraphicsFormat` used for the pipeline's output `RenderTexture`. If not specified, the converter defaults to an ARGB32-compatible format.

Accessing Raw YUV Data & Threading

If you want to handle the raw YUV data yourself, you can skip the converter entirely and create just the session using `CreateContinuousSession()` or `CreateOnDemandSession()`.

You can access the raw frame data by subscribing to the session's native proxy:

```
ContinuousCaptureSession session = cameraDevice.CreateContinuousSession(resolution);
session.NativeProxy.OnFrameReady += (yBuf, ySize, uBuf, vBuf, uvSize, yRowStride, uvRowStride, uvPixelStride,
ts) =>
{
    // Process raw pointers here!
};
```

> **WARNING**
>
> All callbacks triggered by `NativeProxy` (including `OnFrameReady`, `OnClosed`, `OnErred`, etc.) are invoked directly on a background Kotlin/Java thread for maximum performance.

Aborting Captures

If you need to discard all pending and in-progress captures as fast as possible (for example, if a request hangs or the user cancels an action), you can call `CaptureSession.TryAbortCaptures(out ErrorCode errorCode)`.

> ⚠ **WARNING**
>
> **Capture sessions are NOT reusable after aborting.** Once you call `TryAbortCaptures()`, you must dispose of the current pipeline/session and create a new one to capture frames again.

Releasing Resources

Make sure to dispose of all camera resources as soon as possible after you finish using them so the native camera device and capture session are properly closed. You can also force closure synchronously, for example in `OnApplicationQuit`, where Unity won't wait for async methods:

```
Task.WaitAll(
    cameraDevice.DisposeAsync().AsTask(),
    capturePipeline.DisposeAsync().AsTask()
);

Debug.Log("Synchronously closed resources.");
```

Using the `await using` Pattern

If you can use C#'s `await using` statement, you can simplify the entire process significantly. For example:

```
public async Task TakePicture()
{
    if (!QuestCameraManager.Instance.TryGetDevice(CameraInfo.CameraEye.Left, out CameraInfo cameraInfo))
    {
        Debug.LogError("Could not get camera info!");
        return;
    }

    await using CameraDevice cameraDevice = QuestCameraManager.Instance.OpenCamera(cameraInfo);
    if (!await cameraDevice.WaitForInitializationAsync())
    {
        Debug.LogError("Could not open camera!");
        return;
    }

    Resolution resolution = cameraInfo.SupportedResolutions[^1];
    StreamUseCase useCase = cameraInfo.SupportedStreamUseCases.Contains(StreamUseCase.StillCapture)
      ? StreamUseCase.StillCapture : StreamUseCase.None;

    await using CapturePipeline<OnDemandCaptureSession>? capturePipeline =
cameraDevice.CreateOnDemandPipeline(resolution, useCase);
    if (capturePipeline == null || !await capturePipeline.Session.WaitForInitializationAsync())
    {
        Debug.LogError("Could not open capture session!");
        return;
    }

    if (!capturePipeline.Session.TryRequestCapture(out var error))
    {
```

```
        Debug.LogError($"Could not capture frame! {error}");
        return;
    }

    // Wait for the converter callback
    var tcs = new TaskCompletionSource<(RenderTexture, long)>();
    void OnFrame(RenderTexture tex, long ts) => tcs.TrySetResult((tex, ts));
    capturePipeline.Converter.OnFrameProcessed += OnFrame;

    (RenderTexture texture, long timestamp) = await tcs.Task;
    capturePipeline.Converter.OnFrameProcessed -= OnFrame;

    // Process the RenderTexture here!
}
```

Note that an awaiter of `TakePicture` also has to wait for the camera device and pipeline to close.

Vulkan-specific Capture Sessions (Experimental)

If your app uses the Vulkan Graphics API and targets API Level 33 (Android 13) or higher, you can use `VulkanContinuousCaptureSession` and `VulkanOnDemandCaptureSession`, in the `Uralstech.UXR.QuestCamera.Vulkan` namespace, instead of `ContinuousCaptureSession` and `OnDemandCaptureSession`. It can improve memory usage and performance as it uses a native rendering plugin to perform YUV-to-RGBA conversion on the GPU without any CPU copies.

You can create them by calling `CameraDevice.CreateVulkanContinuousSession()`, like so:

```
CameraDevice camera =...;
Resolution resolution =...;

// Create a Vulkan capture session with the camera at the chosen resolution.
VulkanContinuousCaptureSession session = camera.CreateVulkanContinuousSession(resolution,
streamUseCase: StreamUseCase.Preview);
if (!await session.WaitForInitializationAsync())
{
    Debug.LogError("Could not open camera session!");

    // Release camera and session resources - MUST be awaited
    await session.DisposeAsync();
    await camera.DisposeAsync();
    return;
}

// Set the image texture.
_rawImage.texture = session.Texture;
```

For on-demand Vulkan sessions, create it with `camera.CreateVulkanOnDemandSession(Resolution)` and use `await session.ProcessSingleFrameAsync()` to get a captured frame.

OpenGL-specific Capture Sessions

If your app uses the OpenGL Graphics API, you can use `GLESCaptureSession`, in the `Uralstech.UXR.QuestCamera.GLES` namespace, instead of `ContinuousCaptureSession` and `OnDemandCaptureSession`. It can improve memory usage and performance as it uses low-level OpenGL shaders for YUV-to-RGBA conversion on the GPU, without any copies.

It's also simpler to use, as it doesn't require an additional texture converter and provides a read-only `Texture` property (a `Texture2D`) that stores the camera images.

You can create them by calling `CameraDevice.CreateGLESSessionAsync()`, like so:

```
CameraDevice camera =...;
Resolution resolution =...;

// Create a GLES capture session with the camera at the chosen resolution.
GLESCaptureSession session = await camera.CreateGLESSessionAsync(resolution,
streamUseCase: StreamUseCase.Preview);
if (!await session.WaitForInitializationAsync())
{
    Debug.LogError("Could not open camera session!");

    // Release camera and session resources - MUST be awaited
    await session.DisposeAsync();
    await camera.DisposeAsync();
    return;
}

// Start continuous processing
session.StartContinuousProcessing();

// Set the image texture.
_rawImage.texture = session.Texture;
```

For on-demand capture, use `await session.ProcessSingleFrameAsync()` instead of `StartContinuousProcessing()`.

Camera2 Interface

With UXR.QuestCamera v4.1.0, you can now modify the capture requests made by all session types, get all available CameraCharacteristics keys and values from `CameraInfo`, and listen for capture callbacks for all capture requests! A lot of the APIs that enable this are quite similar to their native Camera2 counterparts, so use them only if you're familiar with the Camera2 API.

Get CameraCharacteristics Metadata

By default, `CameraInfo` only exposes a guaranteed subset of the metadata exposed in the wrapped native CameraCharacteristics object. You can access additional metadata through the `TryGet` method:

```
if (cameraInfo.TryGet("CONTROL_AE_AVAILABLE_TARGET_FPS_RANGES", out CameraMetadata.IntRange[]
supportedFpsRanges))
{
    Debug.Log($"Supported FPS ranges: {string.Join(',',
(IEnumerable<CameraMetadata.IntRange>)supportedFpsRanges)}");
}
```

There are some helper methods in `CameraInfo` and its parent `CameraMetadata` to help you with the keys used to access data.

Modify Capture Requests

Immediately after you create a session, add a listener to `session.NativeProxy.ModifyRequestBuilder`. This will be called on a JNI thread, so don't use any Unity APIs here. The events gives you the CaptureRequest Builder and a boolean flag for if the request being built is a repeating or on-demand request. Please see the reference manual for all methods available in `CaptureRequest.Builder`.

This example sets the framerate for a repeating request:

```
session.NativeProxy.ModifyRequestBuilder += static (builder, isRepeatingRequest) =>
{
    if (isRepeatingRequest)
        builder.Set("CONTROL_AE_TARGET_FPS_RANGE", new CameraMetadata.IntRange(15, 15));
};
```

Register Capture Callbacks

Callback events for capture sessions are exclusive to the session's `NativeProxy` and called from a JNI thread. They are not invoked by default due to their performance impact for repeating capture requests. To tell the session to invoke them, add a **single listener** to `session.NativeProxy.ShouldRegisterCaptureEvents` immediately after creating the session, which will return `true` or `false` for capture requests that you want callbacks from. Similar to `ModifyRequestBuilder`, it gives you the `CaptureRequest` and a bool for if it's a repeating request. Now you can actually register listeners to the capture callbacks:

```
_pipeline.NativeProxy.ShouldRegisterCaptureEvents += static (request, isRepeatingRequest) => true;

_pipeline.NativeProxy.OnCaptureCompleted += static (request, result) => Debug.Log($"Capture completed,
frame: {result.GetFrameNumber()}");
_pipeline.NativeProxy.OnCaptureFailed += static (request, failure) => Debug.Log($"Capture
failed: {failure.Reason}");

_pipeline.NativeProxy.OnCaptureSequenceCompleted += static (sequenceId, frameNumber) =>
Debug.Log("Sequence completed.");
_pipeline.NativeProxy.OnCaptureSequenceAborted += static (sequenceId) => Debug.Log("Sequence aborted.");
```

You can obtain the sequence ID of a capture request by listening to `session.OnSessionRequestSetWithId` for repeating requests and GLES sessions and `OnDemandCaptureSession.RequestCapture(CaptureTemplate)`'s `RequestStatus.SequenceId` for on-demand requests.

Example Script

```
using UnityEngine;
using UnityEngine.Android;
using UnityEngine.UI;
using Uralstech.UXR.QuestCamera;

public class CameraTest : MonoBehaviour
{
    [SerializeField] private RawImage _rawImage;

    private CameraDevice _camera;
    private CapturePipeline<ContinuousCaptureSession> _pipeline;

    private async void Start()
```

```
    {
        // Check if the current device is supported.
        if (QuestCameraManager.Support == PCASupport.Unsupported)
        {
            Debug.LogError("Runtime does not support the Passthrough Camera API!");
            return;
        }

        // Check for permission.
        if (!Permission.HasUserAuthorizedPermission(QuestCameraManager.HeadsetCameraPermission))
        {
            Permission.RequestUserPermission(QuestCameraManager.HeadsetCameraPermission);
            return;
        }

        // Get a camera device.
        if (!QuestCameraManager.Instance.TryGetDevice(CameraInfo.CameraEye.Left, out
CameraInfo currentCamera))
        {
            Debug.LogError("No camera available!");
            return;
        }

        // Choose the highest resolution.
        Resolution highestResolution = currentCamera.SupportedResolutions[^1];

        // Open the camera.
        _camera = QuestCameraManager.Instance.OpenCamera(currentCamera);
        if (!await _camera.WaitForInitializationAsync())
        {
            Debug.LogError("Could not open camera!");
            await _camera.DisposeAsync();
            return;
        }

        // Create a capture pipeline with StreamUseCase for best performance
        StreamUseCase useCase = System.Array.Exists(currentCamera.SupportedStreamUseCases, u => u
== StreamUseCase.Preview)
          ? StreamUseCase.Preview : StreamUseCase.None;

        _pipeline = _camera.CreateContinuousPipeline(highestResolution, CaptureTemplate.Preview, useCase);
        if (_pipeline == null || !await _pipeline.Session.WaitForInitializationAsync())
        {
            Debug.LogError("Could not create pipeline!");

            if (_pipeline != null)
                await _pipeline.DisposeAsync();

            await _camera.DisposeAsync();
            return;
        }

        // Set the image texture.
        _rawImage.texture = _pipeline.Converter.Texture;
```

```
        // Dispose later with:
        // await _camera.DisposeAsync();
        // await _pipeline.DisposeAsync();
    }
}
```

Sample - Digit Recognition with Unity Sentis

The package contains a Computer Vision sample that uses an MNIST trained model to recognize handwritten digits, through the Camera API.

Package Dependencies

This sample requires the Unity Sentis (formerly known as Unity Inference Engine (formerly known as Unity Sentis)) package (`com.unity.ai.inference`) and was built with version 2.6.1 of the package.

This sample also uses the old input system. There is no code that references it, but you will have to change the UI input module in the scenes.

Known Issues

YUV Conversion Accuracy

OpenGL's `EXT_YUV_target`[1] and Vulkan's `VK_KHR_sampler_ycbcr_conversion`[2] may produce non-linear R'G'B' values when sampling YUV images. This package applies only rudimentary transfer-function corrections to convert these values to linear RGB, which may result in inaccurate colors.

Both the compute-shader-based YUV converter and the OpenGL converter are potentially inaccurate because they currently assume full-range BT.601 YUV data and do not account for the actual color range or standard of the source image.

[1] https://registry.khronos.org/OpenGL/extensions/EXT/EXT_YUV_target.txt (Issues, 9. Should sampling return samples converted in to linear space or raw samples?)
[2] https://github.com/KhronosGroup/Vulkan-Docs/issues/2356

Advanced Samples

This page contains some samples for advanced use-cases, like custom texture converters or multi-camera streaming.

Custom Texture Converters

The texture converter in `CapturePipeline<T>.Converter` allows you to easily change the conversion compute shader to custom ones. All you have to do is assign a new `ComputeShaderKernel` to `Converter.ShaderKernel`. `ComputeShaderKernel` references the compute shader and the name of the shader kernel to use.

You can also change the compute shader for all new capture sessions by changing `QuestCameraManager.ConversionKernel`.

For example, the following compute shader ignores the U and V values of the YUV stream to provide a Luminance-only image:

```
#pragma kernel CSMain

// ---------------- YUVConverter *required* parameters ----------------

// Camera frame
ByteAddressBuffer YBuffer;
ByteAddressBuffer UBuffer;
ByteAddressBuffer VBuffer;

cbuffer StrideParams {
    // Row strides
    uint YRowStride;
    uint UVRowStride;

    // Pixel strides
    uint UVPixelStride;
    uint StrideParamsPadding;
}

uniform uint OutputTextureWidth;
uniform uint OutputTextureHeight;
RWTexture2D<float4> OutputTexture;

// ----------------     End of required parameters     ----------------

// Converts byte array indexing to ByteAddressBuffer indexing and returns the value
uint GetByteFromBuffer(const ByteAddressBuffer buffer, const uint byteIndex) {
```

```
    const uint word = buffer.Load(byteIndex & ~3);
    const uint byteInWord = byteIndex & 3;

    return (word >> (byteInWord * 8)) & 0xFF;
}

[numthreads(8, 8, 1)]
void CSMain(uint3 id : SV_DispatchThreadID) {

    if (id.x >= OutputTextureWidth || id.y >= OutputTextureHeight)
        return;

    // The YUV stream is flipped, so we have to un-flip it.
    const uint flippedY = OutputTextureHeight - 1 - id.y;

    // Index of Y value in buffer.
    const uint yIndex = flippedY * YRowStride + id.x;

    // Get the Y (luminance) value.
    const uint y = GetByteFromBuffer(YBuffer, yIndex);

    // Convert the value from 0-255 to 0-1 and set the output texture.
    const float3 color = float3(y, y, y) / 255.0;
    OutputTexture[id.xy] = float4(color, 1.0);
}
```

Multiple Streams from One Camera

By adding multiple texture converters to the same request, you can emulate the effect of having more than one image stream from a single camera. For example, you can have one converter stream the camera image as-is, and another streaming with a simple Sepia post-processing effect:

WARNING

This is not the best way to do this! Creating multiple `YUVConverter`s duplicates almost all conversion resources for each instance!

Each `YUVConverter` allocates its own:

- CPU-side `NativeArray` copy buffers
- GPU-side `GraphicsBuffer`s
- Output `RenderTexture`
- Conversion `CommandBuffer`

The CPU-side copy buffers and GPU-side graphics buffers are each approximately the size of one frame of YUV image data. The output `RenderTexture` size depends on the selected texture format, but is usually larger than an equivalent YUV texture.

Please implement a custom converter which shares the above data between different consumers.

```
// Create the session.
_session = _camera.CreateContinuousSession(highestResolution, CaptureTemplate.Preview,
useCase);
if (!await _session.WaitForInitializationAsync())
{
    await _session.DisposeAsync();
    await _camera.DisposeAsync();
    return;
}

// Primary converter, defaults to the shader set in QuestCameraManager.
_primaryConverter = new YUVConverter(highestResolution);

// Secondary converter with the sepia effect, ComputeShaderKernel uses "CSMain" by default.
_secondaryConverter = new YUVConverter(highestResolution, new
ComputeShaderKernel(_sepiaShader));

// Register converters to session.
_session.NativeProxy.OnFrameReady += _primaryConverter.OnFrameReady;
_session.NativeProxy.OnFrameReady += _secondaryConverter.OnFrameReady;

_rawImagePrimary.texture = _primaryConverter.Texture;
_rawImageSecondary.texture = _secondaryConverter.Texture;
```

YUV To RGBA Converter With Sepia Effect

```
#pragma kernel CSMain

ByteAddressBuffer YBuffer;
ByteAddressBuffer UBuffer;
ByteAddressBuffer VBuffer;

cbuffer StrideParams {
    uint YRowStride;
    uint UVRowStride;

    uint UVPixelStride;
    uint StrideParamsPadding;
}

uniform uint OutputTextureWidth;
uniform uint OutputTextureHeight;
RWTexture2D<float4> OutputTexture;

uint GetByteFromBuffer(const ByteAddressBuffer buffer, const uint byteIndex) {

    const uint word = buffer.Load(byteIndex & ~3);
    const uint byteInWord = byteIndex & 3;

    return (word >> (byteInWord * 8)) & 0xFF;
}

[numthreads(8, 8, 1)]
void CSMain(uint3 id : SV_DispatchThreadID) {

    if (id.x >= OutputTextureWidth || id.y >= OutputTextureHeight)
        return;

    const uint flippedY = OutputTextureHeight - 1 - id.y;
    const uint yIndex = flippedY * YRowStride + id.x;

    const uint y = GetByteFromBuffer(YBuffer, yIndex);
    const float3 color = float3(y, y, y) / 255.0;

    float3 sepiaColor;
    sepiaColor.r = dot(color.rgb, float3(0.393, 0.769, 0.189));
    sepiaColor.g = dot(color.rgb, float3(0.349, 0.686, 0.168));
    sepiaColor.b = dot(color.rgb, float3(0.272, 0.534, 0.131));
```

```
    OutputTexture[id.xy] = float4(sepiaColor, 1.0);
}
```

Migrating to UXR.QuestCamera V3

UXR.QuestCamera v3 introduces a soft rewrite of the package with several breaking changes. It's highly recommended to update to v3, as it includes fixes for many crashes and stutters. This page documents all changes to the **public API** in v3.0.0 and v3.1.0 compared to v2.6.1.

Basic Sample

This script shows the most important changes in V3 compared to V2:

```
// Open the camera.
CameraDevice camera = UCameraManager.Instance.OpenCamera(currentCamera);
if (camera == null)
{
    Debug.LogError("Could not open camera!");
    yield break;
}

yield return camera.WaitForInitialization();

// Check if it opened successfully
if (camera.CurrentState != NativeWrapperState.Opened)
{
    Debug.LogError("Could not open camera!");

    // Very important, this frees up any resources held by the camera.

    // V2: camera.Destroy();
    // V3: Yield to DisposeAsync() using extension instead of calling Destroy().
    yield return camera.DisposeAsync().Yield();
    yield break;
}

// Create a capture session with the camera, at the chosen resolution.
// V2: CreateContinuousCaptureSession returns CaptureSessionObject<ContinuousCaptureSession>
// V3: CreateContinuousCaptureSession returns CapturePipeline<ContinuousCaptureSession>

CapturePipeline<ContinuousCaptureSession> sessionObject =
camera.CreateContinuousCaptureSession(highestResolution);
if (sessionObject == null)
{
    Debug.LogError("Could not open camera session!");
    yield return camera.DisposeAsync().Yield(); // V3 dispose
    yield break;
}
```

```
yield return sessionObject.CaptureSession.WaitForInitialization();

// Check if it opened successfully
if (sessionObject.CaptureSession.CurrentState != NativeWrapperState.Opened)
{
    Debug.LogError("Could not open camera session!");

    // Both of these are important for releasing the camera and session resources.

    // V2: sessionObject.Destroy();
    // V3: Yield to DisposeAsync().
    yield return sessionObject.DisposeAsync().Yield();

    yield return camera.DisposeAsync().Yield(); // V3 dispose
    yield break;
}

// Set the image texture.
_rawImage.texture = sessionObject.TextureConverter.FrameRenderTexture;
```

CameraDevice

`CameraDevice` is no longer a `MonoBehavior` and now implements `AndroidJavaProxy` and `IAsyncDisposable`.

- **Removed**
  - `Release()`, `Destroy()` — replaced by `DisposeAsync()`.
  - `IsActiveAndUsable`
- **Changed**
  - `OnDeviceOpened`: `Action<string>` — parameter is the ID of the opened camera.
  - `OnDeviceClosed`: `Action<string?>` — parameter is the ID of the closed camera, or `null` if it failed to open.
  - `OnDeviceErred`: `Action<string?, ErrorCode>` — parameters are the ID of the erred camera (or `null` if it failed to open) and the error code.
  - `OnDeviceDisconnected`: `Action<string>` — parameter is the ID of the disconnected camera.
  - `WaitForInitialization()` now returns a `WaitUntil` object and throws `ObjectDisposedException` if the `CameraDevice` was disposed at the time of calling.
  - `WaitForInitializationAsync()` now accepts an optional `CancellationToken` and throws `ObjectDisposedException` if the `CameraDevice` was disposed at the time of calling.
  - `CreateContinuousCaptureSession()` and `CreateOnDemandCaptureSession()` now return `CapturePipeline<ContinuousCaptureSession>?` and `CapturePipeline<OnDemandCaptureSession>?`,

respectively, and throw `ObjectDisposedException` if the `CameraDevice` was disposed at the time of calling.
    - `CreateSurfaceTextureCaptureSession()` and `CreateOnDemandSurfaceTextureCaptureSession()` now have nullable return types and throw `ObjectDisposedException` if the `CameraDevice` was disposed at the time of calling.
    - `CameraId` is no longer a property and is now a cached value.
- **v3.1.0 Specific Changes**
    - `WaitForInitializationAsync()` now returns `Task<bool>` representing the open state of the device.

CaptureSessionObject

`CaptureSessionObject<T>` has been replaced by `CapturePipeline<T>`, which implements `IAsyncDisposable`.

- **Removed**
    - `GameObject`
    - `CameraFrameForwarder` — functionality moved to `ContinuousCaptureSession`.
    - `Destroy()` — replaced by `DisposeAsync()`.

ContinuousCaptureSession

`ContinuousCaptureSession` is no longer a `MonoBehavior` and now implements `AndroidJavaProxy` and `IAsyncDisposable`.

- **Removed**
    - `Release()` — replaced by `DisposeAsync()`.
    - `IsActiveAndUsable`
- **Changed**
    - `OnSessionConfigurationFailed`: `Action<bool>` — parameter indicates whether the failure was caused by a camera access or security exception.
    - `OnSessionConfigured`, `OnSessionRequestSet`, `OnSessionRequestFailed`: `Action`
    - `WaitForInitialization()` now returns a `WaitUntil` object and throws `ObjectDisposedException` if the `ContinuousCaptureSession` was disposed at the time of calling.
    - `WaitForInitializationAsync()` now accepts an optional `CancellationToken` and throws `ObjectDisposedException` if the `ContinuousCaptureSession` was disposed at the time of calling.
- **v3.1.0 Specific Changes**

- `WaitForInitializationAsync()` now returns `Task<bool>` representing the open state of the session.

OnDemandCaptureSession

`OnDemandCaptureSession` inherits from `ContinuousCaptureSession` and includes the same breaking changes, plus the following:

- **Changed**
  - `RequestCapture()` now throws `ObjectDisposedException` if the `OnDemandCaptureSession` was disposed at the time of calling.

YUVToRGBAConverter

`YUVToRGBAConverter` is no longer a `MonoBehavior` and now implements `IDisposable`.

- **Removed**
  - `Release()` — replaced by `Dispose()`.
  - `OnFrameProcessedWithTimestamp` — replaced with new `OnFrameProcessed`.
  - `SetupCameraFrameForwarder()` — replaced with new constructor `YUVToRGBAConverter(Resolution)`.
  - `CameraFrameForwarder`
- **Changed**
  - `Shader` is now a nullable-aware property.
  - `OnFrameProcessed`: `Action<RenderTexture, long>` — parameters for the frame's `RenderTexture` and the capture's timestamp, in nanoseconds.

SurfaceTextureCaptureSession

`SurfaceTextureCaptureSession` has been moved to the `Uralstech.UXR.QuestCamera.SurfaceTextureCapture` namespace, no longer inherits from `ContinuousCaptureSession`, and now implements `AndroidJavaProxy` and `IAsyncDisposable`.

- **Removed**
  - `Release()` — replaced by `DisposeAsync()`.
  - `Resolution` — use `Texture.width` and `Texture.height` instead.
  - `IsActiveAndUsable`
- **Changed**

    - `Texture` is now a read-only field.
    - `OnSessionConfigurationFailed`: `Action<bool>` — parameter indicates whether the failure was caused by a camera access or security exception.
    - `OnSessionConfigured`, `OnSessionRequestSet`, `OnSessionRequestFailed`: `Action`
    - `WaitForInitialization()` now returns a `WaitUntil` object and throws `ObjectDisposedException` if the `SurfaceTextureCaptureSession` was disposed at the time of calling.
    - `WaitForInitializationAsync()` now accepts an optional `CancellationToken` and throws `ObjectDisposedException` if the `SurfaceTextureCaptureSession` was disposed at the time of calling.
- **v3.1.0 Specific Changes**
    - `WaitForInitializationAsync()` now returns `Task<bool>` representing the open state of the session.

OnDemandSurfaceTextureCaptureSession

`OnDemandSurfaceTextureCaptureSession` (moved to the `Uralstech.UXR.QuestCamera.SurfaceTextureCapture` namespace) inherits from `SurfaceTextureCaptureSession` and includes the same breaking changes, plus the following:

- **Changed**
    - `RequestCapture(Action<Texture2D>)` is now `bool RequestCapture(Action<Texture2D, long>)` where the `long` callback parameter is the capture timestamp, returning the success of the capture, and throws `ObjectDisposedException` if the `OnDemandSurfaceTextureCaptureSession` was disposed at the time of calling.
    - `RequestCapture()` now returns a `WaitUntil?` object (`null` when the capture fails) and throws `ObjectDisposedException` if the `OnDemandSurfaceTextureCaptureSession` was disposed at the time of calling.
    - `RequestCaptureAsync()` is now `Awaitable<(Texture2D?, long)> RequestCaptureAsync()` (`Texture2D` is `null` when the capture fails), accepts an optional `CancellationToken`, and throws `ObjectDisposedException` if the `OnDemandSurfaceTextureCaptureSession` was disposed at the time of calling.
- **v3.1.0 Specific Changes**
    - `RequestCaptureAsync()` now returns `Task<(Texture2D?, long)>`.

CameraInfo

`CameraInfo` is now a record type and implements `IDisposable`.

- **Changed**

- `CameraEye` now defines the following enumeration values:
  - `Unknown = -1`
  - `Left = 0`
  - `Right = 1`
- `CameraSource` now defines the following enumeration values:
  - `Unknown = -1`
  - `PassthroughRGB = 0`
- `LensPoseTranslation` is now nullable (`Vector3?`).
- `LensPoseRotation` is now nullable (`Quaternion?`).
- `Intrinsics` is now nullable (`CameraIntrinsics?`).
- `NativeCameraCharacteristics` is now managed by the `CameraInfo` instance — **do not** dispose it manually.
- `CameraIntrinsics` is now a record type.
- All properties are now read-only fields with cached values.

CameraFrameForwarder (Removed)

`CameraFrameForwarder` has been removed. Its functionality has been moved to `ContinuousCaptureSession`.

- **Changed**
  - `OnFrameReady`: `Action<IntPtr, IntPtr, IntPtr, int, int, int, long>` — moved to `ContinuousCaptureSession`.
    See the `OnFrameReady` documentation for parameter details.

CaptureTemplate

- **Removed**
  - `ZeroShutterLag` — not compatible with capture sessions created by the plugin.

UCameraManager

All methods and properties in `UCameraManager` are now nullable-aware with no breaking changes.